Multi-threading

atomic instruction with retry; independence of the order

Can an operation be atomic without a lock around it?

PART 1

Linearizability and progress

correctness and liveness of a concurrent object

The locked stack example 7a, the reference

Java | OCaml 5

LockedStack.java ↗

```
record Node(int value, Node next) { }
Node top = null;   // under the stack's lock
synchronized void push(int x) {
  top = new Node(x, top);
}
synchronized Integer pop() {
  if (top == null) return null;
  Node old = top;
  top = old.next();
  return old.value();
}
```

```
Java     popped + remaining = 125000500000
OCaml 5  popped + remaining = 125000500000
```

Four threads each push 1 to 250,000 and pop after every second push: what was popped and what remains add up to what was pushed, 125,000,500,000.

Each operation holds the stack's one lock for its whole body: pushes and pops run one after another, each taking effect at one instant inside its critical section. In OCaml, `Mutex.protect m f` holds the lock `m` for the whole of `f`.

One run of each is one sample; ten more runs agreed.

Histories Herlihy and Wing, 1990



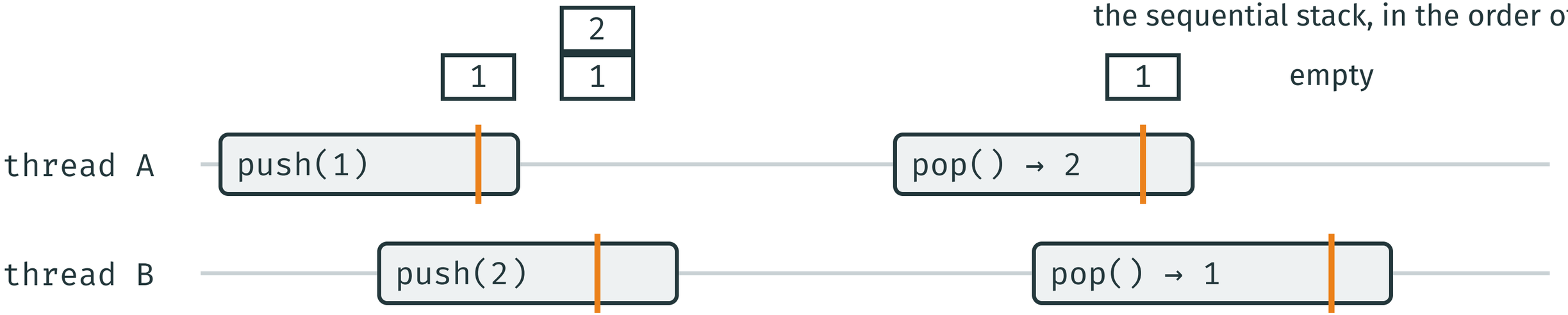
the two pushes overlap: each is called before the other has returned; so do the two pops

A history records one run as seen from outside: for each operation, the thread that called it, which operation it is with its arguments, the instant of its call, the instant of its return, and the value returned.

Two operations **overlap** when each is called before the other has returned.

The sequential stack is the specification: a pop returns the most recent push not yet popped, or `null`.

Linearizability Herlihy and Wing, 1990

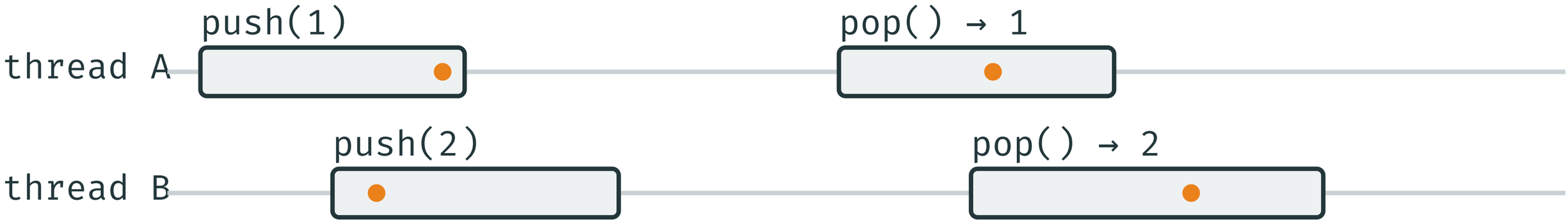


A history is **linearizable**, or explained by the sequential stack, when each operation can be assigned one instant between its call and its return, its **point of effect**, so that the operations taken in the order of these instants form a run of the sequential stack returning what they returned in the history.

An object is linearizable when every history it can produce is.

A history with overlapping pushes

the pops returning 1 and 2

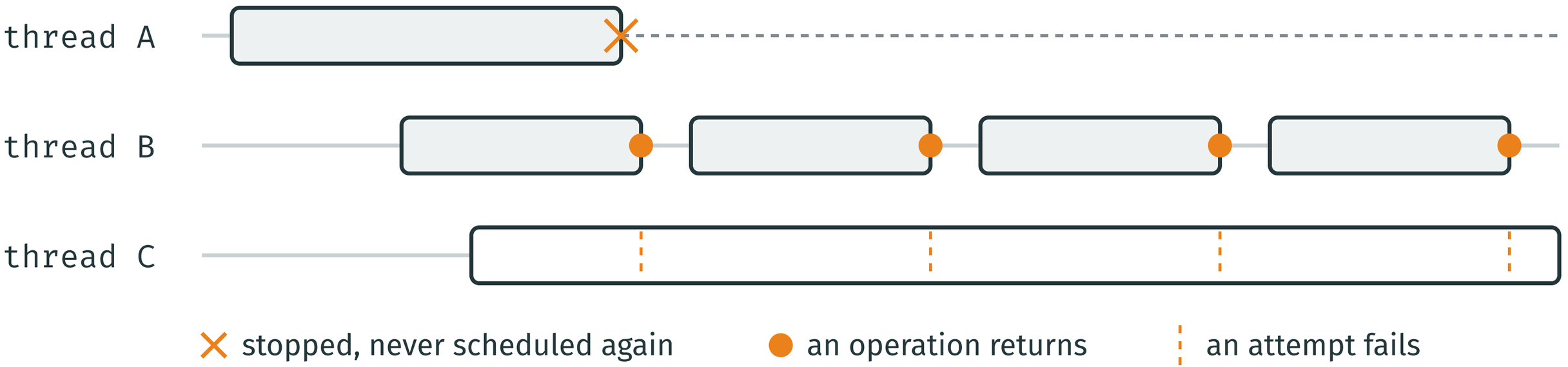


Thread A's pop returns 1, thread B's pop returns 2. Can the sequential stack explain this history?

1. No: `push(1)` was called first, so the first pop must return 2.
2. **Yes: the pushes overlap, so `push(2)` may take effect first; the pops then return 1, then 2.**
3. No: the sequential stack explains no history with overlapping operations.

> No client can tell which push took effect first: the locked stack itself produces this history when thread B takes the lock first.

Progress conditions Herlihy, 1991 and 1993



Blocking: a thread's progress in an operation can depend on another thread's being scheduled. Thread A holds the lock and is stopped: B and C wait for ever.

Lock-free: from any point on, as long as some thread keeps taking steps inside an operation, some operation completes within finitely many steps, whichever threads are stopped and whenever. B keeps completing; each of B's successes fails C's attempt, and C starves.

PART 2

Compare-and-set

an atomic read-modify-write, retried on failure

The counter on compare-and-set example 6

Java | OCaml 5

CounterCas.java ↗

```
// n: an AtomicInteger, from 0
static int increment() {
  int failed = 0;
  while (true) {
    int v = n.get();              // read
    if (n.compareAndSet(v, v+1)) // still v?
      return failed;              // wrote v+1
    failed++;                     // retry
  }
}
```

```
Java     got 2000000, failed attempts 653147
OCaml 5  got 2000000, failed attempts 679563
fetch-and-add: got 2000000 in both
```

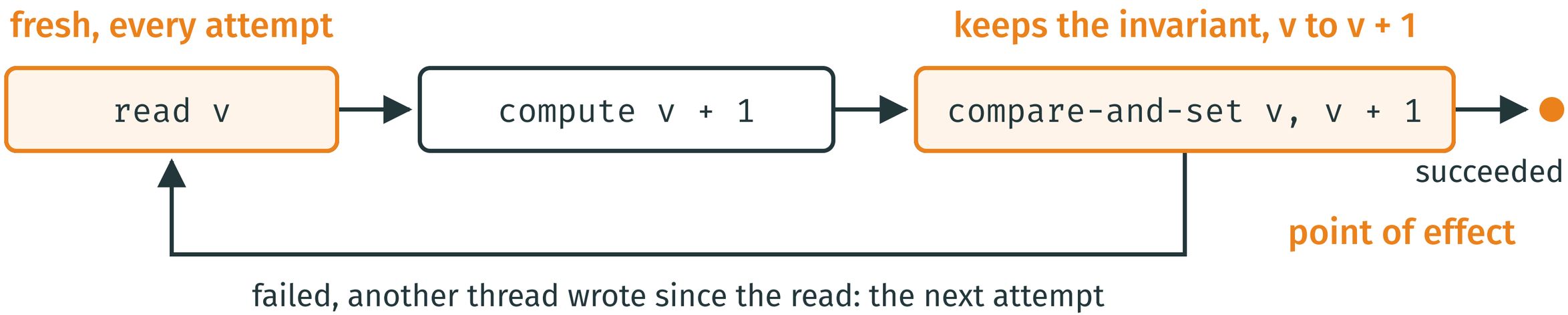
Two threads, a million calls each. Can the total be wrong, and can an attempt fail?

1. exact; no attempt ever fails
2. **exact; an attempt fails when another thread wrote since its read**
3. wrong whenever an attempt fails

A failed attempt loses no increment: the loop reads again. Fetch-and-add, the last line, never retries.

Compare-and-set with retry

the second way, in the four terms

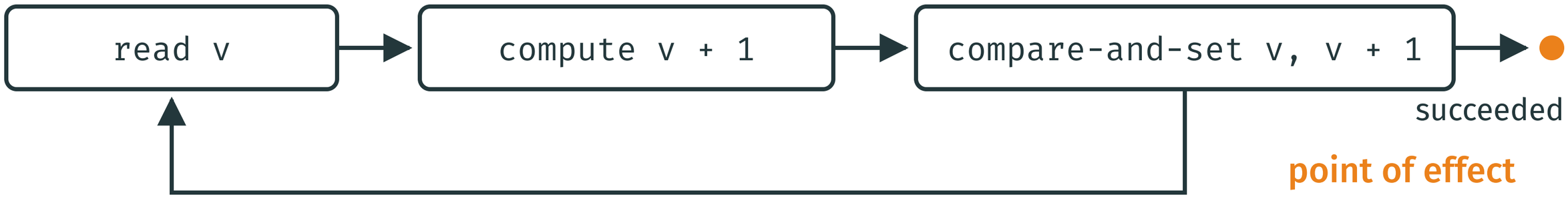


The contract. The operation is atomic without exclusion, and linearizable: its point of effect is the compare-and-set that succeeds.

The obligation. Every attempt starts from a fresh read; every write keeps the invariant, here that it replaces v by v + 1; the programmer names the point of effect.

Compare-and-set with retry

the progress risk; who checks



no check: a loop that keeps a stale read compiles and runs

The progress risk. A compare-and-set fails only because another succeeded, so operations keep completing: lock-free. Not wait-free, which would bound every thread's own steps: a thread may fail at every attempt while the others finish. Fetch-and-add asks for no expected value, so it never fails: wait-free, each call one step.

Who checks. Nobody but the programmer, by proof; Java and OCaml accept a loop that keeps an old value, or two atomic steps meant to be one.

PART 3

Treiber's stack

a lock-free stack on one atomic reference

Treiber's stack

example 7b: push and pop as compare-and-set loops

Java | OCaml 5

Treiber.java ↗

```
record Node(int value, Node next) { }

static void push(int x) {
  while (true) {
    Node old = top.get();
    Node fresh = new Node(x, old);
    if (top.compareAndSet(old, fresh))
      return;  // point of effect: the CAS
  }
}
static Integer pop() {
  while (true) {
    Node old = top.get(); // effect if empty
    if (old == null) return null;
    if (top.compareAndSet(old, old.next()))
      return old.value(); // effect: the CAS
  }
}
```

`top` is an atomic reference; the nodes are immutable, a record's fields being set once. The invariant: `top` points to a well-formed stack, a chain of nodes ending in `null`, in OCaml a list.

A push builds its node outside the stack, pointing at the top it read, and replaces `top` by it with a compare-and-set, which succeeds only if `top` is still the one it read; otherwise it reads again and builds a new node.

```
Java     popped + remaining
         = pushed 125000500000
OCaml 5  popped + remaining
         = pushed 125000500000
```

A push without compare-and-set

a read, then a write

Java | OCaml 5

Treiber.java ↗

```
static void pushBroken(int x) {
  Node old = top.get();          // read
  top.set(new Node(x, old));     // write
}
```

```
Java     on the stack 307454 of 1000000 pushed
OCaml 5  on the stack 350362 of 1000000 pushed
```

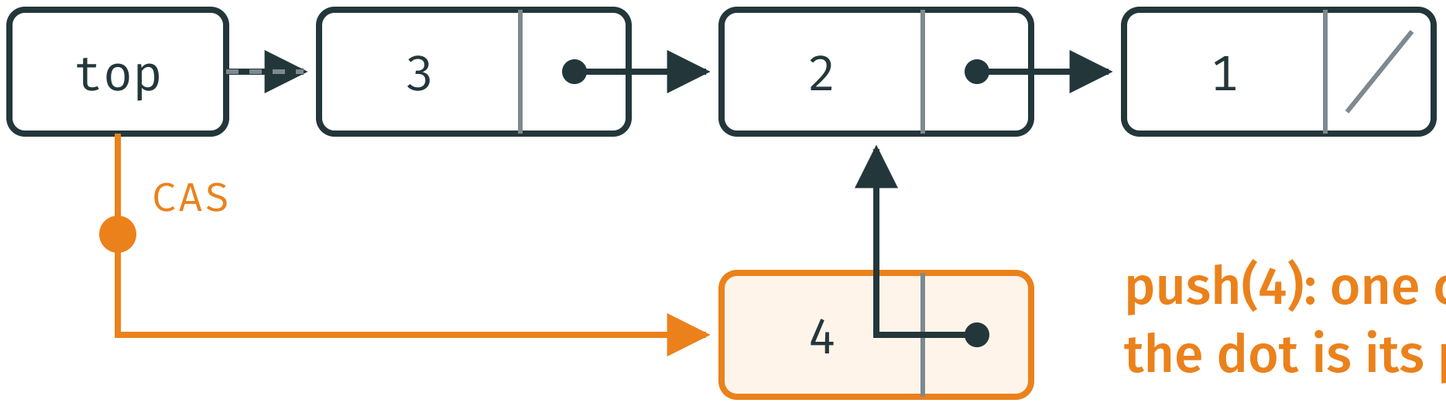
Four threads push only, a million pushes in all. Where does it go wrong, and is it a data race?

1. **between its read and its write another push took effect, and the write loses it; not a data race, both accesses being atomic**
2. the read and the write are a data race with the other threads' writes, since nothing orders them
3. the immutable nodes are the problem: a node cannot be re-pointed at a new top

The bug is the racy counter's: a step of another thread falls between a thread's read and the write that depends on it. An atomicity violation without a data race.

Proof sketch for Treiber's stack

the invariant; the points of effect



push(4): one compare-and-set on top; the dot is its point of effect

The invariant. `top` points to a well-formed stack, the list of values from the top node down to `null`. It holds at the start; every write to `top` is a compare-and-set that keeps it; a failed attempt writes nothing.

The points of effect. For a push, the compare-and-set that succeeds. For a pop on a non-empty stack, the same. For a pop on an empty stack, the read that saw `top` null.

Proof sketch for Treiber's stack safety; liveness

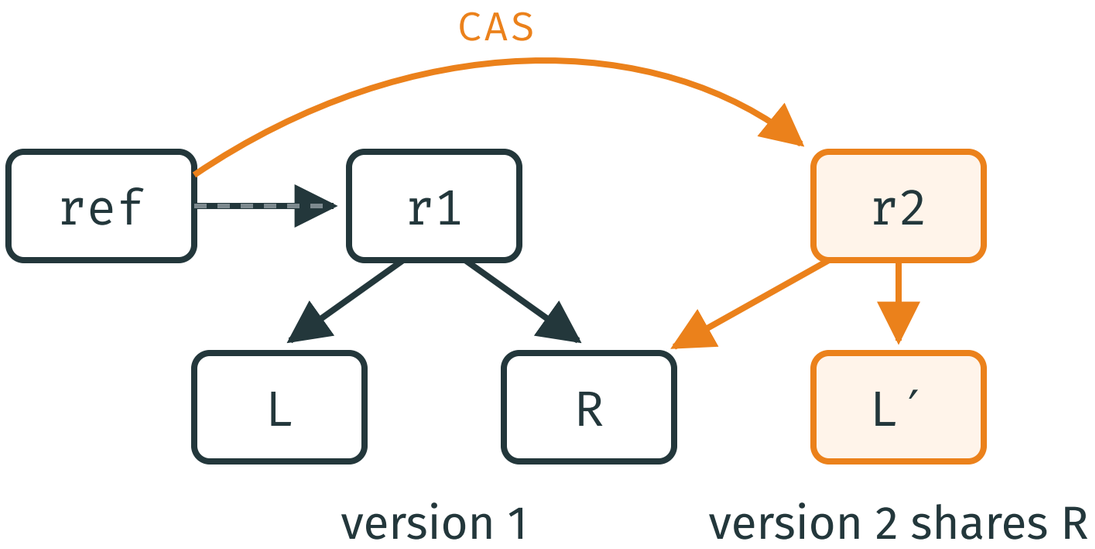


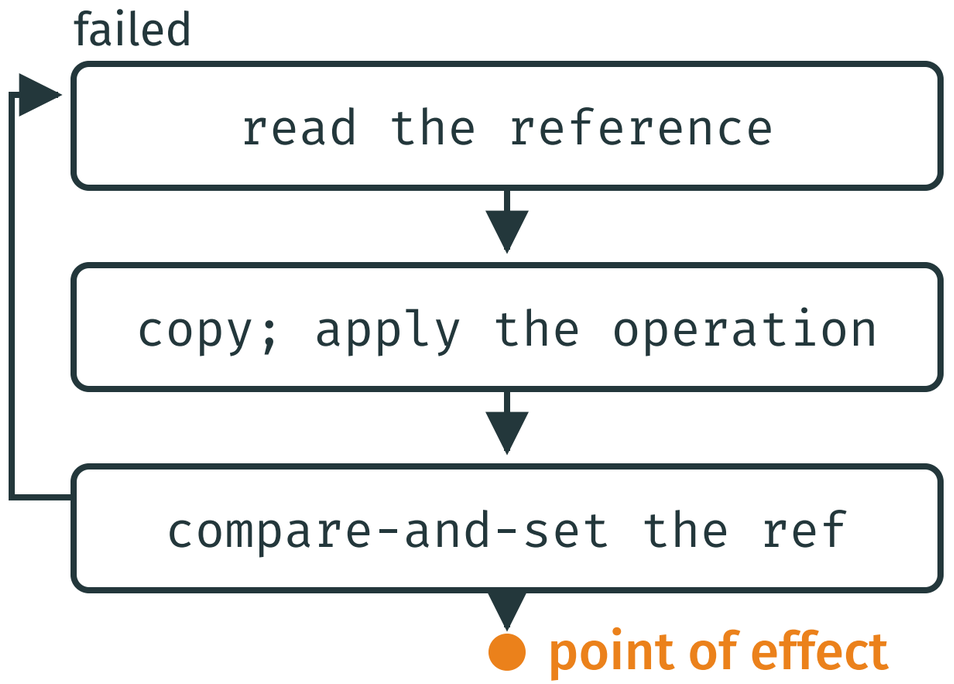
A's attempt fails: B's compare-and-set succeeded since A's read; A re-reads and succeeds

Safety. In the order of the points of effect, the operations form a run of the sequential stack: between two of them `top` does not change, and each does one sequential push or pop and returns what it returns.

Liveness. An attempt fails only because another thread's succeeded since its read, and each success completes an operation; so operations keep completing: lock-free, and not wait-free.

Persistent data structures and compare-and-set Herlihy, 1993

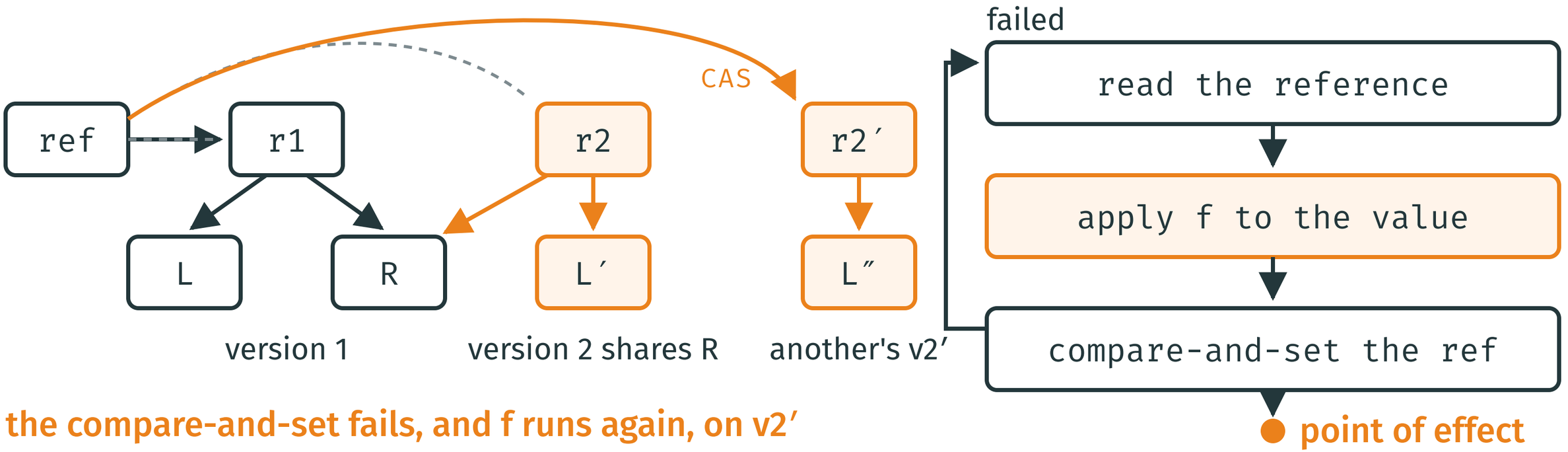




A persistent structure keeps its old version usable after an update; the new version shares the unchanged part. Treiber's stack is such a value behind one atomic reference, and a read of the reference is a snapshot of the whole stack.

Herlihy, 1993. Read the reference, apply the operation to a copy, swing the reference to the copy with a store-conditional, which fails if anything wrote since the read; retry on failure.

Persistent data structures and compare-and-set Clojure's atom



the compare-and-set fails, and f runs again, on v2′

Clojure's `swap!` applies a function to the value in an atom, an atomic reference, by that protocol on compare-and-set: read, apply, compare-and-set, retry.

Under contention the function runs several times: in three runs, 800,000 swaps ran it between 4.2 and 4.6 million times. So it must have no effects, and nothing checks it. Java's `updateAndGet` is the same loop with the same warning.

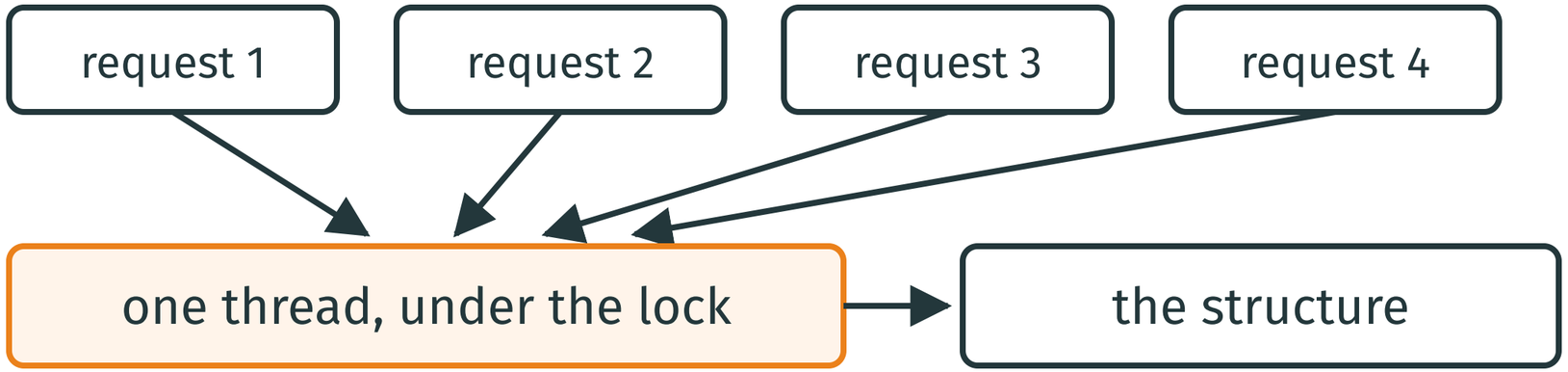
PART 4

Contention

the cost of retry

Lock-free against lock-based contention

flat combining



one thread performs every request; the others publish theirs and wait

Contention. Several threads want the same word at once: one attempt succeeds, the rest are wasted work, and every operation on the stack passes through the top. An uncontended lock costs one step, like one attempt.

Flat combining. Each thread publishes its request; one thread, under a coarse lock, performs them all: lock-based, blocking, and faster (Hendler, Incze, Shavit and Tzafrir, 2010).

PART 5

Deterministic parallelism

disjoint parts and an associative combine

A count per thread, then the sum

example 8 (a), the counter's third version

Java | OCaml 5

PerThread.java ↗

```java
final int N = 1_000_000;    // per thread
long[] counts = new long[2];
Thread[] ts = new Thread[2];
for (int t = 0; t < 2; t++) {
  final int id = t;
  ts[t] = new Thread(() -> {
    long mine = 0;              // local
    for (int i = 0; i < N; i++) mine++;
    counts[id] = mine;
  });
}
for (Thread t : ts) t.start();
for (Thread t : ts) t.join();
System.out.println(counts[0] + counts[1]);
```

```
Java     2000000
OCaml 5  got 2000000
```

Each thread counts in a variable of its own; the order in which the threads run cannot matter, because no location written by one thread is read or written by another while they run.

The argument rests on the `join`, which orders each thread's write of its element of `counts` before the main thread's read. Without it the program has a data race and may print 0, one million or two million.

In Rust the compiler checks that nothing is shared, by giving each slot one owner; in Haskell the types do, since a pure function cannot write a cell. In Java, nobody does.

The parallel fold

example 8 (b) and (c): 1 to 8 from 0, with + and with −

A fold combines the elements of a sequence with an operation, one after another from a starting value, the seed. Here the sequence 1 to 8 is cut into parts; each part is folded from the seed 0 on a thread of its own; after the joins the parts' results are combined with the same operation, in the order of the parts.

```
+ in two halves 36, in four quarters 36
- in two halves 16, in four quarters  0
Java and OCaml 5 print the same
```

The sequential fold with subtraction, one thread from left to right, gives −36. Which is true of the parallel fold?

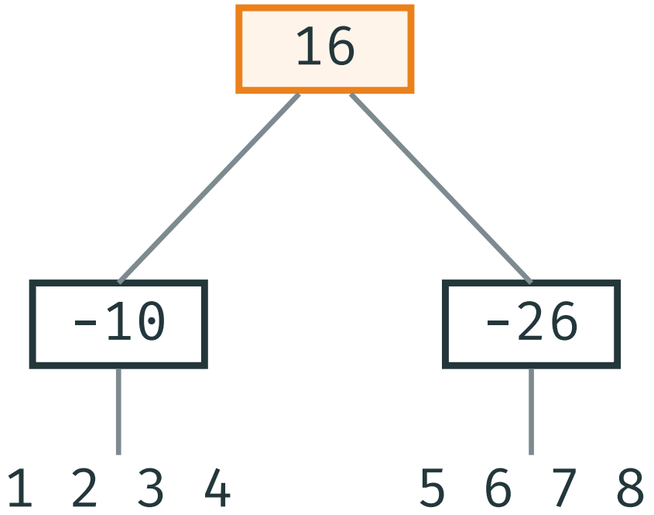
1. it agrees in two halves and in four quarters, and the answer is the same on every run: the threads only share the work
2. **it agrees in neither, and the answer is the same on every run**
3. it agrees in neither, and the answer changes from run to run: it depends on which thread finishes first

With subtraction the answer depends on how the sequence is split, never on which thread finishes first. If the operation is associative, every part is folded from its identity, and the parts are combined in their order, every split gives the sequential fold's answer.

Two bracketings of one fold

subtraction on 1 to 8

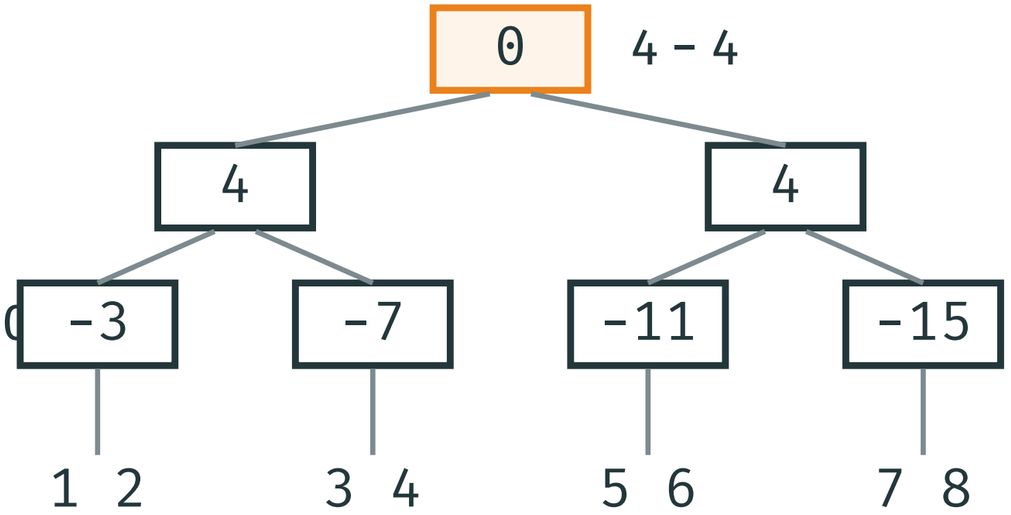
two halves



−10 − (−26)

each part folded from (

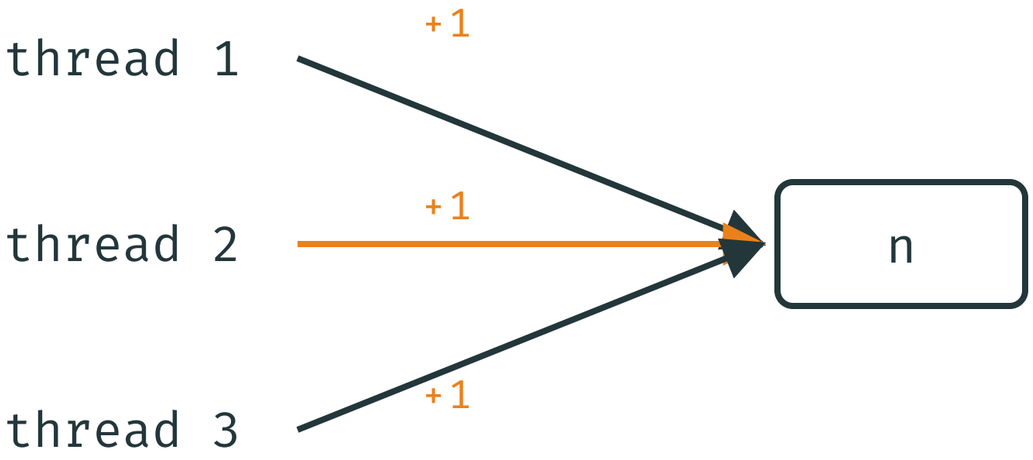
four quarters



The parts are the same numbers in the same order; only the bracketing differs, and the roots differ. With addition both roots are 36, as is the sequential fold, because addition is associative.

The forms of the third way

independent parts; commuting updates



one atomic step each: any order gives the same total

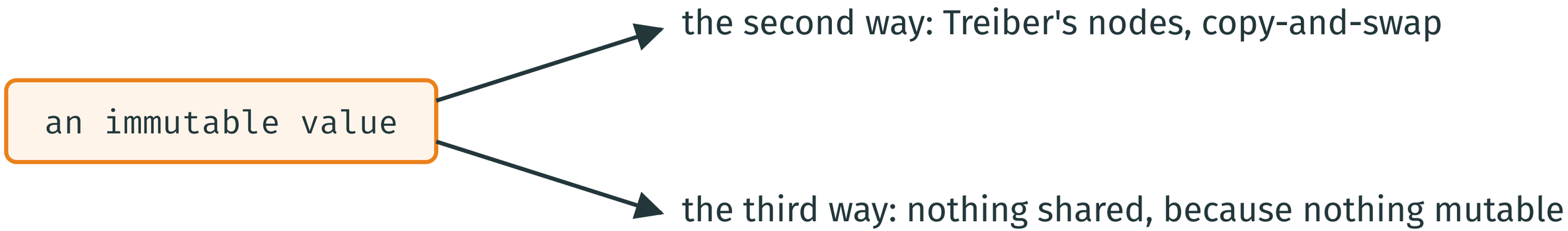
or a replica per thread, merged at the end

Independent parts and a combine step. No location written by one thread is read or written by another while they run; the combine is associative with an identity, and the parts are combined in their order.

Commuting updates. Each update is one atomic step, or goes to a replica merged at the end; the interleaving orders the steps, and commutativity makes the final state the same for every order.

The forms of the third way

determinism by construction; immutability



not a fourth way

Determinism by construction. A dataflow network, sequential programs connected by queues with one writer and one reader each, computes the same function in every schedule (Kahn, 1974); single-assignment variables do the same.

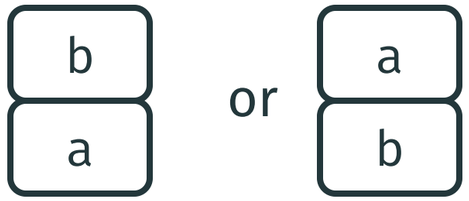
Immutability is not a fourth way: it serves the second, Treiber's nodes and copy-and-swap, and the third, nothing shared because nothing mutable.

The counter and the stack compared

which work the third way is for

```
thread 1 pushes a

thread 2 pushes b
```



the pops show which came first

no single intended result; linearizability suffices

The counter. Increments commute: applying them in any order gives the same total. So when each increment is atomic, under a lock or by one instruction, or is made in a slot of its own, every way gives the same final value.

The stack. Two pushes on a shared stack do not commute, and the pops show which came first: the result depends on the interleaving under every way; the only demand on it is linearizability.

PART 6

Composition

the atomicity of two operations combined

Composition example 4 (a), the transfer in two halves

Java | OCaml 5

Accounts.java ↗

```
static void transferA(Account from,
                      Account to, int x) {
  synchronized (from) { from.balance -= x; }
  synchronized (to)   { to.balance   += x; }
}
```

```
Java     first wrong reading: A = 99, B = 100
OCaml 5  first wrong reading: A = 234, B = -35
```

The two balances add up to 200 between transfers. A reader holding both locks sees 199: it read between the two halves of a transfer. Two atomic steps are not one atomic step, under locks or under compare-and-set.

Two linearizable objects used together are a linearizable system, each operation at its own instant: Herlihy and Wing's locality. What is not given is one instant for two operations: the withdrawal and the deposit each have theirs.

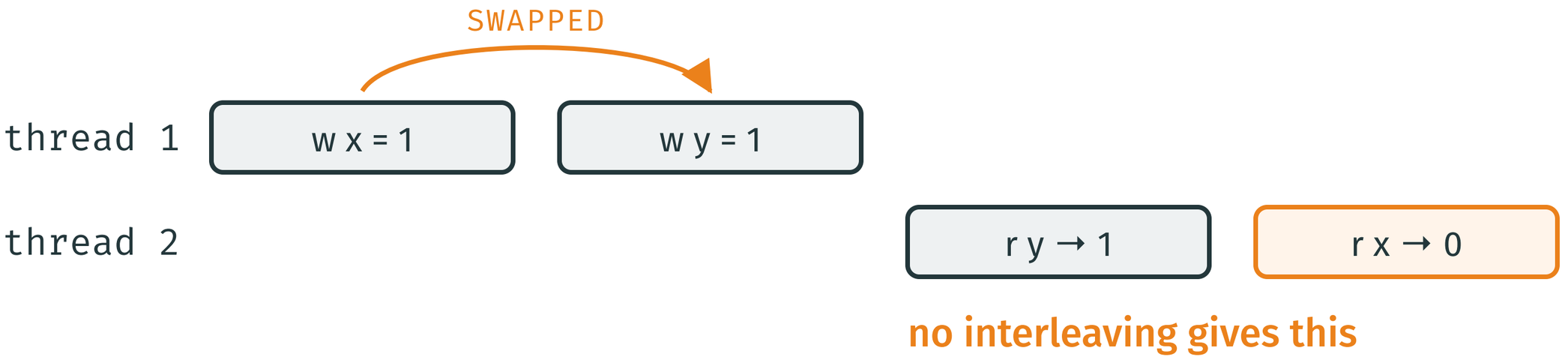
PART 7

Memory models

sequential consistency for data-race-free programs

The memory model's guarantee

the fourth assumption, with its condition



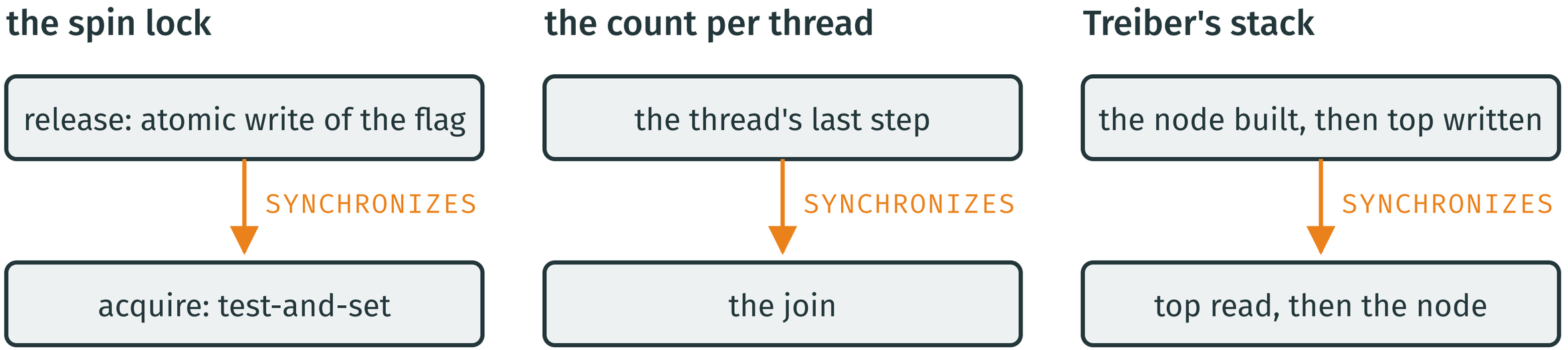
no interleaving gives this

race-free programs still run as one sequence: Adve and Hill, 1990, DRF-SC

There is one memory, and every read returns the value of the latest write before it in the interleaving (Lamport, 1979: sequential consistency). The language guarantees it **for every program none of whose interleavings has a data race**, with its atomics in the strongest mode.

Why a guarantee. Processor and compiler reorder reads and writes for speed. Adve and Hill, 1990: a race-free program still runs as one sequence; the language's **memory model** says so.

The memory model's guarantee the session's programs



every run was some interleaving, and the reasoning by interleavings was sound

What made the session's programs race-free. In the spin lock, the order test-and-set gives between one thread's release and the next's acquire; in the count per thread, the join; in the loop and Treiber's stack, every access to `top` atomic, the nodes published through it.

The guarantee applied to them: every run was some interleaving, and the reasoning by interleavings was sound.

The cost of a data race

example 4 (d) of block 1, the reader with no lock

Java | OCaml 5

Accounts.java ↗

```
static int[] readUnlocked() {   // no lock
  return new int[] { A.balance, B.balance };
}
```

```
Java      138670249 readings, 20954124 wrong
OCaml 5   109518919 readings, 2611258 wrong
```

The transfers take their locks in a global order; the reader takes none. Its two reads are each a data race. What does the language guarantee for it?

1. nothing: a racy program has no meaning in any language
2. every run is some interleaving: the transfers are locked
3. **no guarantee of an interleaving; what remains is set by each language's memory model, and in Java and OCaml 5 a race is bounded**

C and C++: undefined behaviour. Java: a read returns a value once written to that variable, bounded in space. OCaml 5: also bounded in time. Go: like Java for one word. Safe Rust: no data race can be written.

PART 8

Summary

the three ways compared

The ways compared

The way	Contract	Obligation	Progress risk	Who checks
Mutual exclusion: forbid the interleaving	atomic by exclusion; linearizable by construction	every access under the lock; the invariant; a global order of the locks	deadlock, starvation; a stopped holder stops the threads that wait for its lock	nobody (Java, C, C++); a detector at run time (Go, ThreadSanitizer); the compiler (Rust, data races)
Atomic instruction with retry: detect it and retry	atomic without exclusion; linearizable at a point of effect	the loop re-reads; the invariant; the point of effect named	lock-free, not wait-free: a thread may retry for ever; fetch-and-add wait-free	the programmer, by proof; the collector removes ABA when no node is reused; checkers of histories
Independence of the order: make it irrelevant	a deterministic final result	independent parts and an associative combine in order; or commuting atomic updates; or determinism by construction	none from interleaving; the join waits for every worker	that nothing is shared, the type system can; associativity or commutativity, the programmer only

Conclusion of block 2

The session's question. Three questions of any program: which contract it chooses, how that contract is stated, and which obligation is left to the programmer.

The lab builds three structures in Java, each with its contract and a check: the counter in three versions, worker threads, and the bounded buffer, with a barrier as the extension. Then it makes a small ray tracer multi-threaded with them, so that each mistake shows in the picture.

Workers. The thread that starts the workers owns them and joins them. Stopping is a request through a flag, which ends nothing by itself.

The mini-project builds the bounded buffer as a queue in three versions, with one lock, with two locks and without locks, behind one interface, with a checker of histories and a measurement of throughput.

Session 8: no sharing at all, by messages.

Session 9: transactions, snapshot isolation, and the structures that merge replicas.

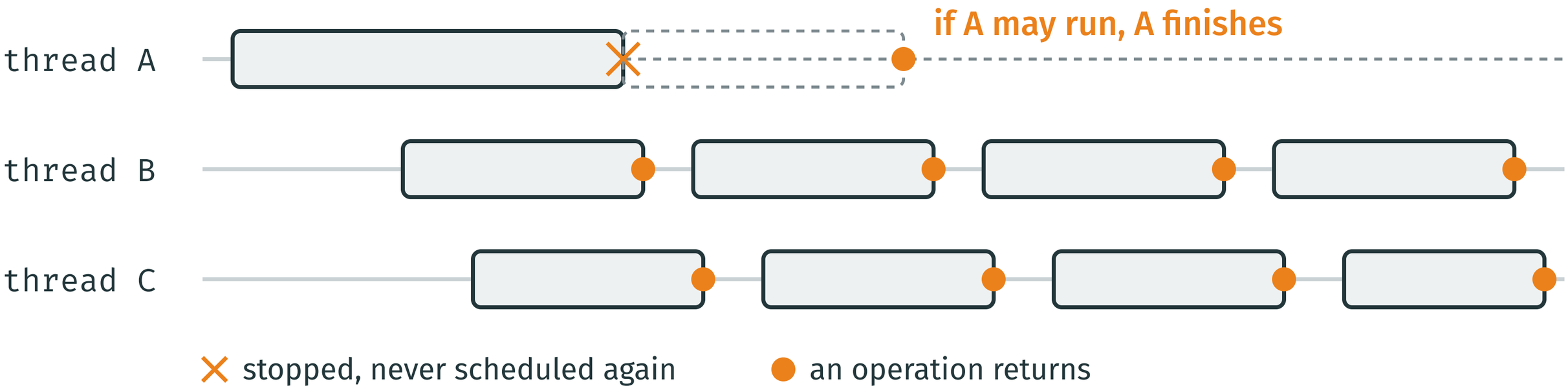
Session 12: types for concurrency.

APPENDIX

Progress, universality and composition

wait-free; the ABA problem; the whole structure; consensus numbers; transactions; the languages compared

Progress conditions wait-free; the clause



Wait-free: every thread that keeps taking steps inside an operation completes it within a finite number of its own steps, whichever other threads are stopped and whenever. B and C both complete.

The clause "whichever threads are stopped" is what makes these conditions about progress: without it a lock would count as lock-free, since a holder that is allowed to run finishes.

Construction and progress

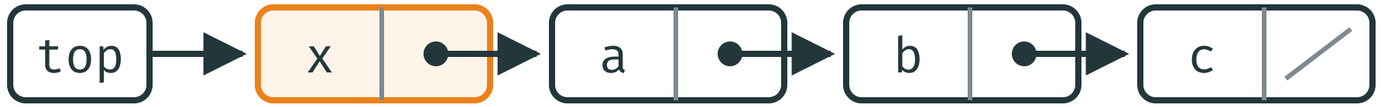
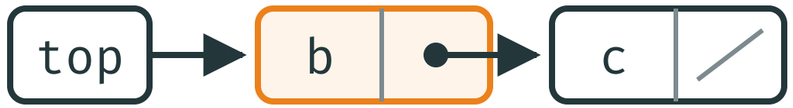
		non-blocking	
	blocking	lock-free	wait-free
lock-based	spin lock; synchronized	impossible	impossible
without locks	**the Disruptor**	the loop; Treiber's stack	fetch-and-add

Lock-based and **without locks** say how a structure is built; **blocking**, **lock-free** and **wait-free** say what progress it guarantees, and nothing else. A lock-based structure is never lock-free.

A structure built without locks is not thereby lock-free: the Disruptor, a queue over a ring buffer, has no lock on its path and still blocks.

The ABA problem

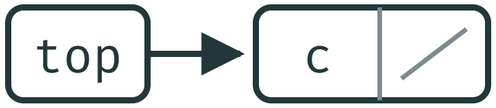
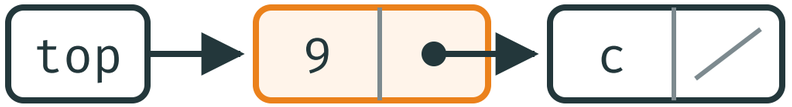
the hazard; on this stack

thread 1, in pop	thread 2	the stack after the event
reads top = a, next = b		
	pushes x	
	pops x	 same node a
compare-and-set a → b: succeeds		 right

The hazard. A thread reads a location as A; other threads change it to B and back to A; the thread's compare-and-set, expecting A, succeeds although the structure it reasoned about may have changed.

On this stack. A push and a pop between the read and the compare-and-set leave the very node read at the top again: harmless, since that node, never changed, heads the same chain.

The ABA problem with a collector; without one

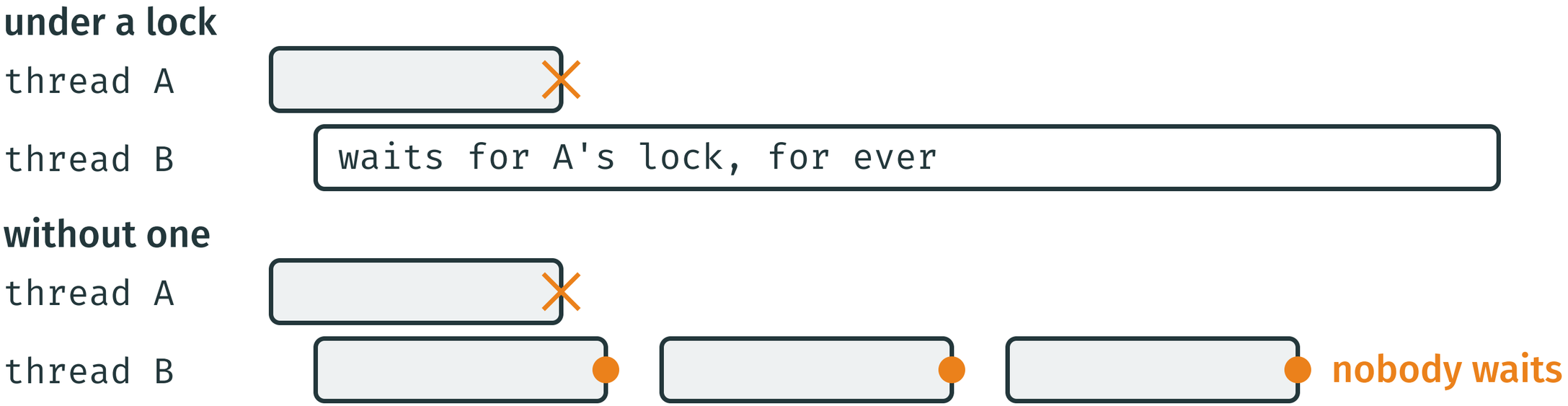
thread 1, in pop	thread 2	the stack after the event
reads top = a, next = b		
	pops a and b: both freed	 a and b freed
	pushes 9 at a's address	 the address a again
compare-and-set a → b: succeeds		 **top = b, freed: corrupted**

The garbage collector. A node any thread still holds is never freed, so its address is never reused, and every push allocates a fresh node: a different node at the same address cannot arise.

Without a collector. In C or C++ a popped node may be freed and its memory reused; a pop that read the old node follows a `next` into freed memory and corrupts the stack. Hazard pointers and epochs delay the freeing of a node until no thread can still hold it.

Lock-free against lock-based

the whole structure; the progress guarantee

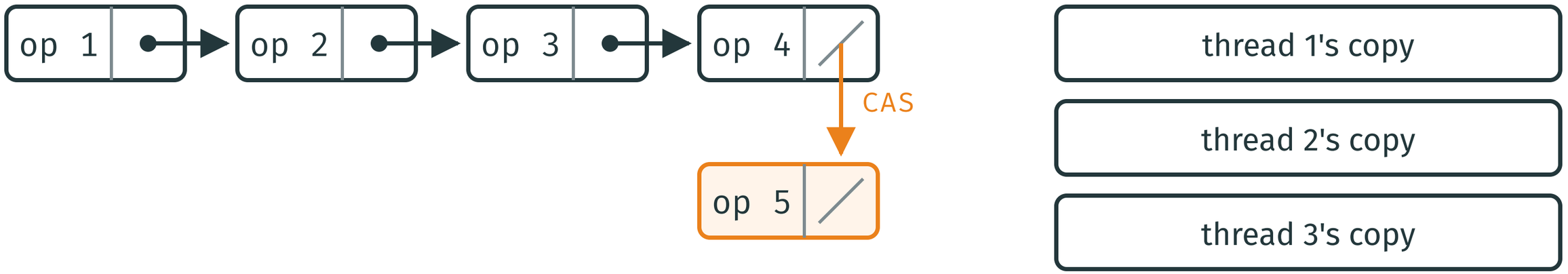


Operations on the whole structure. Where nodes change in place there is no instant at which the whole is unchanged: Java's `ConcurrentLinkedQueue.size` traverses and may be inaccurate. A persistent value behind one reference has that instant, the read of the reference.

Progress guarantee. A stopped lock holder stops every waiter; a thread stopped in mid-push stops nobody. Where that must hold, only a non-blocking structure will do.

Universality of compare-and-set Herlihy, 1991

the log of all operations

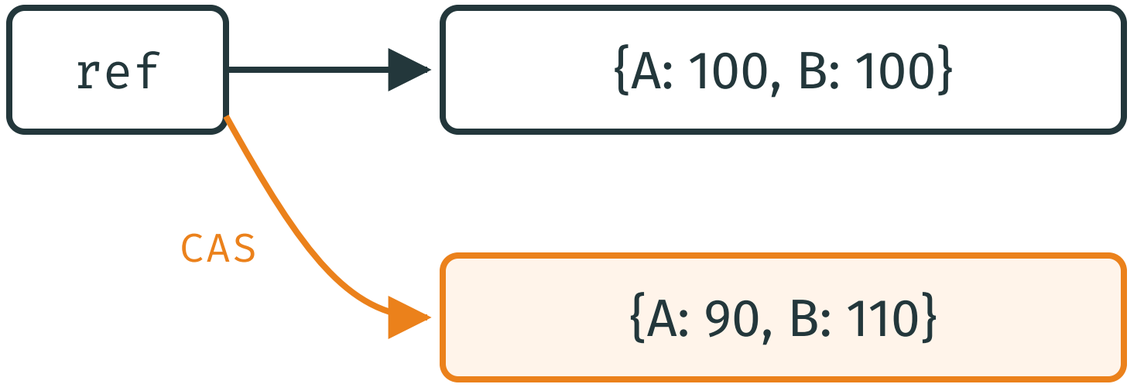


append: one compare-and-set on the last node's next; every thread replays the log on its copy

Every structure. With compare-and-set, every object with a sequential specification has a wait-free, linearizable version; with reads and writes alone, not even a stack that two threads share.

The construction. Every operation is appended to one shared list, which every thread applies in the same order to a copy of its own; a thread that appends also appends the operation of the thread whose turn it is. Nothing runs in parallel.

Ways to compose locks; one reference



the transfer: one compare-and-set

every update of the map competes for this one reference

Locks compose by hand. Both locks around both steps make the transfer one operation, at the cost of deadlock and a global order of the locks.

One compare-and-set covers one reference. Both accounts behind one reference to a persistent map: the transfer is one compare-and-set, and every update of the map competes for that reference, the cost of Clojure's atom.

Ways to compose transactions

Clojure, eight threads, 800,000 transfers, the course's run

a reader inside a transaction	**never wrong, in 4,352,597 reads**
a reader outside	wrong 5,345 times

Haskell: the counter under atomically exact, ten runs of ten

Transactions compose, lock-based operations do not (Harris et al., 2005). A transaction's writes take effect together, at one instant, or not at all; on a conflict the runtime reruns it.

In the course's runs, Haskell's counter under `atomically` was exact in ten runs of ten; in Clojure, with 800,000 transfers in one `dosync` each, a reader inside a transaction was never wrong and a reader outside one was wrong 5,345 times.

Languages with shared memory Java, OCaml 5, Rust

Language	What the programmer expresses	Who checks	What a race costs	What holds by construction
Java	locks, atomics with four modes, `volatile`, `join`; all three ways	nobody in the language or the JDK	defined; a value once written to that variable; bounded in space, not in time	a correctly synchronized program runs as an interleaving
OCaml 5	`Mutex`, `Condition`, `Atomic`, `Domain.join`; all three ways	nothing in the compiler; a ThreadSanitizer build at run time	defined; bounded in space and in time	a race-free program, and every race-free location, runs as an interleaving
Rust	ownership; a `Mutex<T>` owning its data; atomics with a mode per access	the compiler, data races only	cannot be written in safe Rust	shared or mutable, never both, unless a lock or an atomic is between

Languages with shared memory, continued

Go, C and C++, Haskell

Language	What the programmer expresses	Who checks	What a race costs	What holds by construction
Go	`sync.Mutex`, atomics, channels; sharing allowed, the proverb advises against it	the race detector at run time	one word like Java; several words, memory corruption	a race-free program runs as an interleaving (2022)
C, C++	`std::mutex`, `std::atomic` with a mode per access	nobody; ThreadSanitizer at run time	undefined behaviour	race-free, with mutexes and default atomics: an interleaving
Haskell	pure functions; mutation in typed cells; `atomically`	the types: a pure function cannot mutate; a transaction cannot do I/O	a read then a write on an `IORef` loses updates, as the racy counter	an ordinary value cannot change; transactions compose

Languages that limit shared mutation Erlang, Clojure, JavaScript

Language	What the programmer expresses	Who checks	What a race costs	What holds by construction
Erlang	processes that share nothing and send copied messages; shared tables for speed	nothing to check between processes	no data race between processes; a get then a set by messages still loses updates	a process's state is its own, one message at a time
Clojure	immutable values; an atom (`swap!`); refs with `dosync`; agents	nothing: the function of `swap!` should be pure, and nothing enforces it	the function of `swap!` runs once per attempt; its effects repeat	an atom's value replaced as a whole; a transaction sees a snapshot
JavaScript	workers that receive copies; `SharedArrayBuffer` and `Atomics` by choice	nothing	a plain `++` on a shared array loses updates; `Atomics.add` does not	without a shared buffer nothing is shared; one thread per worker